Lecture 26: Internet Control Management Protocol, ICMP (RFC 792)

Objectives:

- Learn the basics of ICMP
- Learn how to construct packet headers using Raw sockets
- Learn how to write a Ping and TraceRoute applications using ICMP and raw sockets.

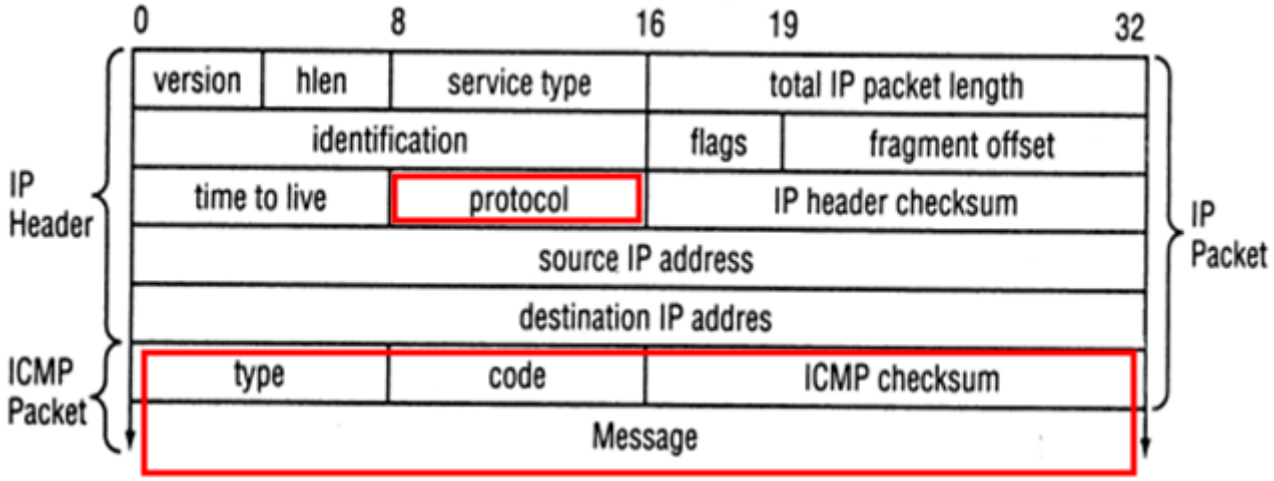
1. Overview of ICMP

Internet Control Message Protocol (ICMP), is a protocol that is used to allow network devices to report errors and other conditions in data transmission.

Like TCP and UDP, ICMP uses IP to communicate across network. Also like UDP, ICMP uses connectionless approach, so packet delivery is unreliable.

Recall that IP packets identify the next layer protocol contained in the data section using the *protocol type* field. ICMP packets are identified with protocol type value of *1*.

The following figure shows how ICMP packet fields are placed in an IP packet:



1.1 Type and Code fields:

There are different types of messages that ICMP packet can carry. These different messages are grouped into types.

The 1-byte ***type*** field in used to specify the type of message that is enclosed in the packet.

Some of the types are further divided into sub-types. The next 1-byte ***code*** field is used to specify the sub-type.

The following table shows some of the types and some of the codes.

Type	Code	Description
0	0	for echo reply message (also see Type 8)
3	0	net unreachable
3	1	host unreachable
3	2	protocol unreachable
3	3	port unreachable
3	4	fragmentation needed and DF set
3	5	source route failed
3	6	destination network unknown
3	7	destination host unknown
3	8	source host isolated
3	9	communication with destination network administratively prohibited
3	10	communication with destination host administratively prohibited
3	11	network unreachable for type of service
3	12	host unreachable for type of service
4	0	source quench message
5	0	Redirect datagrams for the Network
8	0	for echo request message (see Type 0)
11	0	time to live exceeded in transit
12	0	pointer indicates the error (identifies the octet where an error was detected.)
13	0	for timestamp message
14	0	for timestamp reply message
15	0	for information request message
16	0	for information reply message

1.2 Checksum:

The 2-byte *checksum* is used to ensure that the packet has arrived without corruption.

The checksum is computed based on the ICMP portion of the packet, using a specific algorithm defined in RFC792 (discussed below).

1.3 Message:

The message part is a variable size component that represents the message being sent.

The message part contains various other fields that are unique to individual ICMP message types.

Many of the ICMP message types define the first two fields in the message as an identifier and a sequence number. The two fields are used to uniquely identify a message.

The following shows the format of the packet for echo request and reply:



2. Using Raw Sockets

Because ICMP packets do not use either TCP or UDP, we cannot use the socket helper classes (UdpClient, TcpListener, TcpClient) to transmit them. Instead, we have to use raw sockets.

Raw sockets allow us to define our own network packet above the IP layer. This means, we need to do all the work of creating the individual fields ourselves.

To create a raw socket, we must use the SocketType.Raw when creating the socket.

Following are some of the ProtocolType values that can be used with SocketType.Raw:

Value	Description
Ggp	Gateway-to-Gateway Protocol
Icmp	Internet Control Message Protocol
Idp	Xerox Internet Datagram Protocol
Igmp	Internet Group Management Protocol
Ipx	Novell IPX Protocol
Spx	Novell SPX Protocol
Raw	Raw IP packet

The specific protocol for a raw socket allows the .NET library to properly create the underlying IP packet – give the proper protocol type value.

For example, by using ProtocolType.Icmp, the IP packet created will have the IP protocol type field set to ICMP (value 1).

```
Socket socket = new Socket(AddressFamily.InterNetwork,
                           SocketType.Raw, ProtocolType.Icmp);
```

2.1 Sending Raw Packets:

Because ICMP is connectionless, sending ICMP packet is just like sending UDP packets – no need to bind the socket to a specific local end point.

Also ICMP does not require any specific port for sending as the packets are received at the IP layer label. The target application will then read the ICMP part of the message directly.

However, since the constructor of the IPEndPoint class requires a port number, 0 (meaning any port) is normally used as shown below:

```
IPEndPoint remoteEP = new
            IPEndPoint(IPAddress.Parse("192.168.1.2"),  0);
socket.SendTo(packet, remoteEP);
```

2.2 Receiving Raw Packets:

Receiving data from a raw socket:

Receiving data from a raw socket is also similar to receiving UDP packets. However, since raw socket does not specify any high-level protocol, the whole packet (including the IP headers) is received.

Thus, to extract the ICMP packet, we must start reading the byte array received, starting from byte number 20.

3. Creating an ICMP class

Since the Raw socket does not create an ICMP packet, we need to create a class to do that. The following describes how such a class may be created.

Fields:

The class will have an instance variable to represent each of the fields in the ICMP packet, namely, *type*, *code*, *checksum* and *message*.

Constructors:

We need at least two constructors for the class – one for creating an ICMP packet from scratch and the other for re-constructing the ICMP packet from a received array of bytes.

Methods:

To send an ICMP packet, the packet has to be converted to an array of bytes.

Thus, we need a method, `public byte[] getBytes()`, that will convert the ICMP packet into a single array of bytes.

When a packet is received, a checksum value must be computed and compared against the value stored in the checksum field.

If the two values do not match, then the packet has some errors and should therefore be rejected.

Thus, we need a method, **`public UInt16 getChecksum()`**, that computes the checksum of the ICMP packet.

According to RFC 792, the checksum is computed as "the 16-bit one's complement of the one's complement sum of the ICMP message, starting with the ICMP type, where the checksum field is set to zero."

To be able to implement the above algorithm, one needs the knowledge of binary arithmetic, which we did not cover in this course.

Thus, we simply use the method provided in the following full listing of the ICMP class.

```
using System;
using System.Net;
using System.Text;

public class Icmp
{
   private byte type;
   private byte code;
   private UInt16 checksum;
   private byte[] message = new byte[1024];
   private int messageSize;


   public Icmp()
   {
   }

   public Icmp(byte type, byte code, UInt16 checksum,
                           byte[] message, int messageSize) {
      this.type = type;
      this.code = code;
      this.checksum = checksum;
      this.message = message;
      this.messageSize = messageSize;
   }

   public Icmp(byte[] data, int size)
   {
      type = data[20];
      code = data[21];
      checksum = BitConverter.ToUInt16(data, 22);
      messageSize = size - 24;
      Buffer.BlockCopy(data, 24, message, 0, messageSize);
   }
```

```
   public byte[] getBytes()
   {
      byte[] data = new byte[messageSize + 5];
      Buffer.BlockCopy(BitConverter.GetBytes(type), 0, data, 0, 1);
      Buffer.BlockCopy(BitConverter.GetBytes(code), 0, data, 1, 1);
      Buffer.BlockCopy(BitConverter.GetBytes(checksum),0,data, 2,2);
      Buffer.BlockCopy(message, 0, data, 4, messageSize);
      return data;
   }

   public UInt16 getChecksum()
   {
      UInt32 checksum = 0;
      byte[] data = getBytes();
      int packetSize = messageSize + 4;
      int index = 0;

      while ( index < packetSize) {
         checksum += Convert.ToUInt32(BitConverter.ToUInt16(data,
index));
         index += 2;
      }
      checksum = (checksum >> 16) + (checksum & 0xffff);
      checksum += (checksum >> 16);
      return (UInt16)(~checksum);
   }

   public byte Type {
      get {return type;}
      set {type = value;}
   }

   public byte Code {
      get {return code;}
      set {code = value;}
   }

   public UInt16 Checksum {
      get {return checksum;}
      set {checksum = value;}
   }

   public byte[] Message {
      get {return message;}
      set {message = value;}
   }
   public int MessageSize {
      get {return messageSize;}
      set {messageSize = value;}
   }
}
```

Notice that the size of the data array in the *getBytes* method is one more than the actual data.

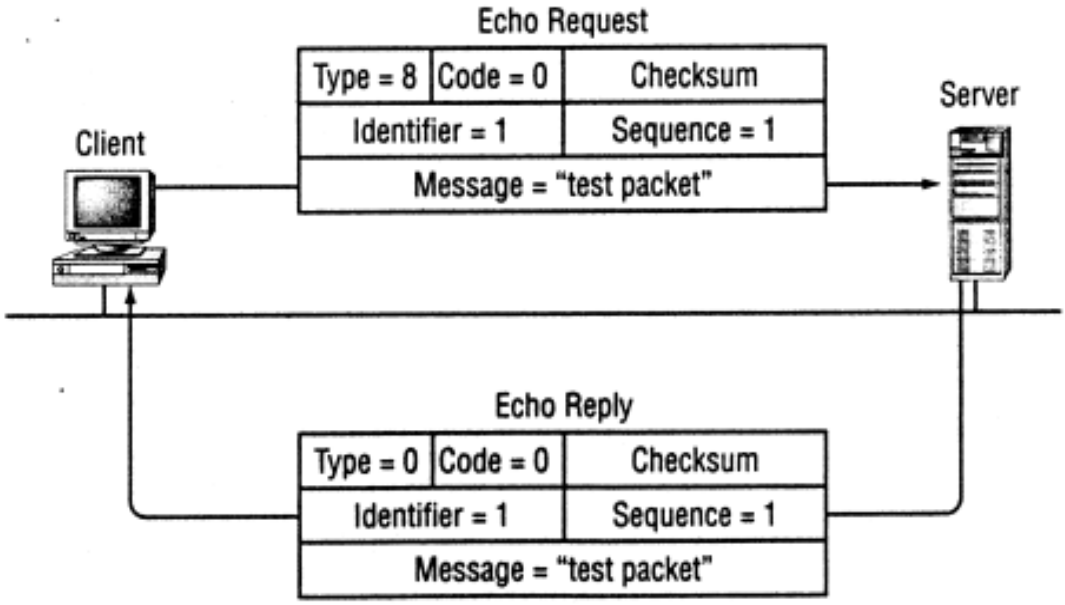
The purpose of the extra one byte is to ensure the correct computation of the checksum value even when the message size is odd.

4. Writing a Ping Application

One of the common use of ICMP is writing an application that checks the reachability of a host on the network.

Such an application is known as Ping. Ping makes use of ICMP type 8 (echo request) and type 0 (echo reply).

The client sends a message with a specific identification and sequence number to some target machine and the target machine returns the same message back. The following figure shows this setup.



The following shows how to write such an application:

```
using System;
using System.Windows.Forms;
using System.Net;
using System.Net.Sockets;
using System.Text;
using System.Threading;
```

```
namespace ICMPApplications
{
   class MyPing : System.Windows.Forms.Form
   {
      private System.Windows.Forms.Label label2;
      private System.Windows.Forms.Button btStart;
      private System.Windows.Forms.TextBox host;
      private System.Windows.Forms.TextBox result;
      private System.Windows.Forms.TextBox databox;
      private System.Windows.Forms.Label label;
      private System.Windows.Forms.Button btClose;
      private System.Windows.Forms.Button btStop;

       private static int pingstart, pingstop, elapsedtime;
       private static Thread pinger;
       private static Socket socket;

      public MyPing()
      {
         InitializeComponent();
          socket = new Socket(AddressFamily.InterNetwork,
SocketType.Raw, ProtocolType.Icmp);
          socket.SetSocketOption(SocketOptionLevel.Socket,
SocketOptionName.ReceiveTimeout, 5000);
      }

      void btStartClick(object sender, System.EventArgs e)
      {
          result.Text = "";
           pinger = new Thread(new ThreadStart(SendPing));
           pinger.IsBackground = true;
           pinger.Start();
      }

       void SendPing()
        {
         IPHostEntry remoteHost = Dns.Resolve(host.Text);
         EndPoint remoteEP = new
IPEndPoint(remoteHost.AddressList[0], 0);

         Icmp packet = new Icmp();
         int recv, i = 1;

         packet.Type = (byte) 8;
         packet.Code = (byte) 0;
         Buffer.BlockCopy(BitConverter.GetBytes(1), 0,
packet.Message, 0, 2);
        byte[] data = Encoding.ASCII.GetBytes(databox.Text);
         Buffer.BlockCopy(data, 0, packet.Message, 4,
data.Length);
         packet.MessageSize = data.Length + 4;
         int packetsize = packet.MessageSize + 4;
```

```
        result.Text += "Pinging " + host.Text+"";
        while(true)
         {
           packet.Checksum = 0;
           Buffer.BlockCopy(BitConverter.GetBytes(i), 0,
packet.Message, 2, 2);
           UInt16 chcksum = packet.getChecksum();
           packet.Checksum = chcksum;

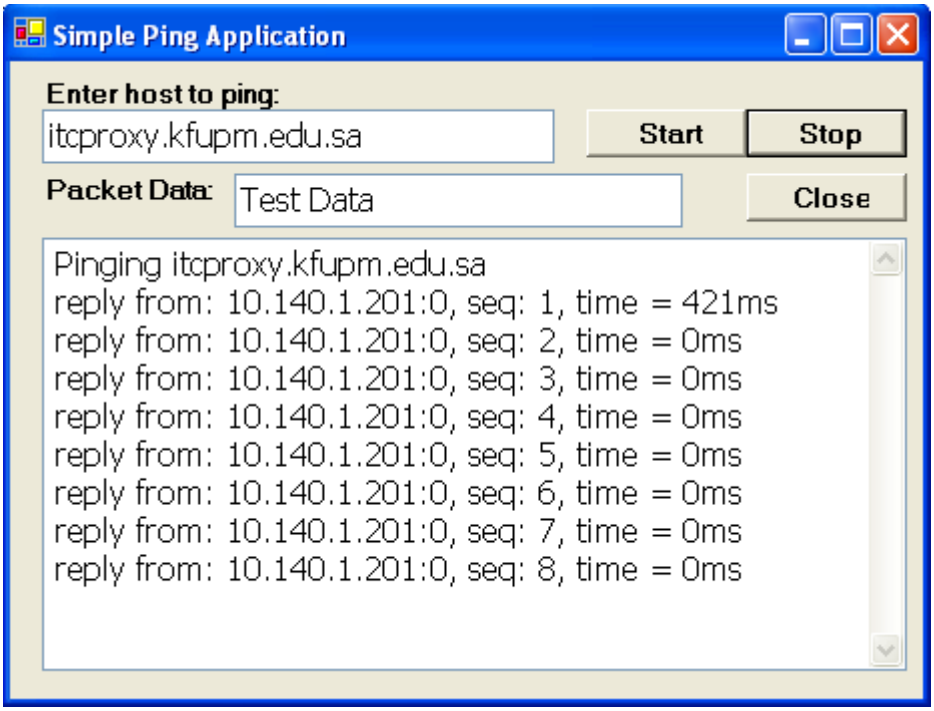
           pingstart = Environment.TickCount;
           socket.SendTo(packet.getBytes(), packetsize,
SocketFlags.None, remoteEP);
           try
           {
              data = new byte[1024];
              recv = socket.ReceiveFrom(data, ref remoteEP);
              pingstop = Environment.TickCount;
              elapsedtime = pingstop - pingstart;
              result.Text += "reply from: " +
remoteEP.ToString() +
                 ", seq: " + i +  ", time = " + elapsedtime +
"ms";
            } catch (SocketException)
            {
              result.Text += "no reply from host";
            }
           i++;
           Thread.Sleep(2000);
         }
       }

     void btStopClick(object sender, System.EventArgs e){
          pinger.Abort();
     }

     void btCloseClick(object sender, System.EventArgs e) {
          socket.Close();
          Close();
     }

     void InitializeComponent() {
        //deleted;
     }

     [STAThread]
     public static void Main(string[] args){
        Application.Run(new MyPing());
     }
   }
}
```



5. Writing a Trace Route Application

Another application of ICMP is trace-route. That is finding the exact route that a packet followed before reaching its target.

The trace-route application sends an ICMP echo request to a remote host. However, it sets the TTL value by increasing sequence of value – starting from one.

Each time the TLL value falls to zero without reaching the target, the packet dies out, and the router at that point is expected to send an ICMP packet of type 11 (Time Exceeded).

By displaying the sending address of each time-exceeded packet, we can watch each router along the path of the ping packet.

The following shows the trace-route application.

```
using System;
using System.Windows.Forms;
using System.Net;
using System.Net.Sockets;
using System.Text;
using System.Threading;

namespace ICMPApplications {
   class MyTraceRoute : System.Windows.Forms.Form  {
      private System.Windows.Forms.Label label2;
```

```
      private System.Windows.Forms.Button btClose;
      private System.Windows.Forms.Button btStart;
      private System.Windows.Forms.TextBox host;
      private System.Windows.Forms.Label label;
      private System.Windows.Forms.TextBox databox;
      private System.Windows.Forms.Button btStop;
      private System.Windows.Forms.TextBox result;

       private static int timestart, timestop, elapsedtime;
       private static Thread tracer;
       private static Socket socket;

      public MyTraceRoute()     {
         InitializeComponent();
          socket = new Socket(AddressFamily.InterNetwork,
SocketType.Raw, ProtocolType.Icmp);
          socket.SetSocketOption(SocketOptionLevel.Socket,
SocketOptionName.ReceiveTimeout, 3000);
      }

      void btStartClick(object sender, System.EventArgs e)
      {
          result.Text = "";
            tracer = new Thread(new ThreadStart(TraceRoute));
            tracer.IsBackground = true;
            tracer.Start();
      }

       void TraceRoute()
        {
          IPHostEntry remoteHost = Dns.Resolve(host.Text);
          IPEndPoint remoteEP = new
IPEndPoint(remoteHost.AddressList[0], 0);
          EndPoint dummyEP = (EndPoint) remoteEP;

          Icmp packet = new Icmp();
          int recv, i;

          packet.Type = (byte) 8;
          packet.Code = (byte) 0;
          packet.Checksum = 0;
          Buffer.BlockCopy(BitConverter.GetBytes(1), 0,
packet.Message, 0, 2);
          Buffer.BlockCopy(BitConverter.GetBytes(1), 0,
packet.Message, 2, 2);
          byte[] data = Encoding.ASCII.GetBytes(databox.Text);
          Buffer.BlockCopy(data, 0, packet.Message, 4, data.Length);
          packet.MessageSize = data.Length + 4;
          int packetsize = packet.MessageSize + 4;

          UInt16 chcksum = packet.getChecksum();
          packet.Checksum = chcksum;
```

```
        result.Text += "Tracing " + host.Text+"";

        int badCount = 0;
        for (i=1; i<50; i++)
         {
           socket.SetSocketOption(SocketOptionLevel.IP,
                                  SocketOptionName.IpTimeToLive, i);
          timestart = Environment.TickCount;
           socket.SendTo(packet.getBytes(), packetsize,
SocketFlags.None, remoteEP);

           try
           {
              data = new byte[1024];
              recv = socket.ReceiveFrom(data, ref dummyEP);
              timestop = Environment.TickCount;
              elapsedtime = timestop - timestart;

             Icmp response = new Icmp(data, recv);
             if (response.Type == 11)
                result.Text += "hop "+i+ ": respnse from "+
dummyEP.ToString()+", "+elapsedtime +"ms";
            else if (response.Type == 0) {
                result.Text += dummyEP.ToString()+ " reached in
"+elapsedtime+ "ms.";
                break;
             }
             badCount=0;
            } catch (SocketException)
            {
              result.Text += "hop "+i+": No response in
"+elapsedtime+ "ms.";
             badCount++;
             if (badCount == 5) {
                result.Text += "Unable to contact remote host";
                break;
             }
            }
           Thread.Sleep(1000);
         }
       }

     void btStopClick(object sender, System.EventArgs e)  {
          tracer.Abort();
     }

     void btCloseClick(object sender, System.EventArgs e)
     {
          socket.Close();
          Close();
     }
```

```
      void InitializeComponent() {
         //deleted
      }

      [STAThread]
      public static void Main(string[] args)
      {
         Application.Run(new MyTraceRoute());
      }
   }
}
```

